

Wetterpilot 2.0 – Detaillierte Spezifikation (Implementierungsstand, v6)

Änderung: Theme für Dark Mode (Felder statt Grau → dunkleres Blau)

Ziel: Alle Felder/Container, die im Dark Mode bislang grau wirkten, erhalten eine **dunkelblaue Fläche**, die bewusst **dunkler als der Rahmen** ist. So bleiben Rahmen/Abgrenzungen sichtbar, die Flächen integrieren sich in die Navy-Ästhetik und wirken nicht mehr „fremd-grau“.

Farbdefinitionen (Theme.swift)

- `navyBase` = #0A183A (Primärer App-Hintergrund, Navigation, ForecastCard-Basis)
- `navyFrame` = #0F2A64 (Rahmen/Linien, etwas heller als Surface für Kontrast)
- `navySurface` = #07102A (Flächen für Felder, **dunkler** als Rahmen)

Regeln

- Dark Mode:
- App-Grundfläche = `navyBase`.
- **Feldflächen** (Form-/List-Row, TextField-Container, Sektionen) = `navySurface`.
- **Rahmen/Border** um Felder = 1pt Stroke in `navyFrame` (oder Weiß 12–20% für subtile Linien).
- Text/Icons auf allen Flächen = Weiß.
- Light Mode:
- Unverändert zu v5 (weiß/hellgrau), ForecastCards bleiben immer dunkelblau.

Stil-Multiplikatoren

- CornerRadius 14–16 pt, Innenabstand 10–12 pt (konsistent zu ForecastCard).
- Schatten im Dark Mode minimal oder gar nicht (Flächen wirken sonst grau).

SwiftUI – Code-Vorgaben (Theme.swift)

- Neue Farben: `AppColor.navyBase`, `AppColor.navySurface`, `AppColor.navyFrame`.
- Neue Modifier:
- `.fieldContainer()` → dunkle Fläche + Border für Input-Zeilen/Container.
- `.listRowNavy()` → setzt `listRowBackground(AppColor.navySurface)`.

Anwendung

- View1Start:
- Jede Etappen-Zeile (Datum + TextField) als Container mit `.fieldContainer()`.
- Für List/Section: `.listRowNavy()` statt transparenter Hintergründe.
- View3Summary/View4Detail:
- Unverändert: `.forecastCard()` (immer dunkelblau).

Beispiel-Code (Theme.swift)

```
import SwiftUI
```

```
enum AppColor {  
    static let navyBase = Color(red: 10/255, green: 24/255, blue: 58/255) // #0A183A  
    static let navySurface= Color(red: 7/255, green: 16/255, blue: 42/255) // #07102A (dunkler als  
    Rahmen)  
    static let navyFrame = Color(red: 15/255, green: 42/255, blue:100/255) // #0F2A64  
    static let onNavy = Color.white
```

```
  
    static func text(for scheme: ColorScheme) -> Color {  
        scheme == .dark ? .white : AppColor.navyBase  
    }  
    static func bg(for scheme: ColorScheme) -> Color {  
        scheme == .dark ? AppColor.navyBase : .white  
    }  
}
```

```
  
struct AppBackground: ViewModifier {  
    @Environment(\.colorScheme) private var scheme  
    func body(content: Content) -> some View {  
        content  
        .foregroundColor(AppColor.text(for: scheme))  
        .scrollContentBackground(.hidden)  
        .background(AppColor.bg(for: scheme).ignoresSafeArea())  
    }  
}  
extension View { func appBackground() -> some View { modifier(AppBackground()) } }
```

```
  
struct ForecastCard: ViewModifier {  
    func body(content: Content) -> some View {  
        content  
        .foregroundColor(AppColor.onNavy)  
        .padding(12)  
        .background(RoundedRectangle(cornerRadius: 14, style: .continuous).fill(AppColor.navyBase))  
    }  
}  
extension View { func forecastCard() -> some View { modifier(ForecastCard()) } }
```

```
  
struct FieldContainer: ViewModifier {  
    @Environment(\.colorScheme) private var scheme  
    func body(content: Content) -> some View {  
        content  
        .padding(10)  
        .background(  
            RoundedRectangle(cornerRadius: 14, style: .continuous)  
            .fill(scheme == .dark ? AppColor.navySurface : Color(UIColor.secondarySystemBackground))  
            .overlay(RoundedRectangle(cornerRadius: 14, style: .continuous)
```

```

.stroke(scheme == .dark ? AppColor.navyFrame.opacity(0.8) : Color(UIColor.separator), lineWidth:
1))
)
}
}
extension View {
func fieldContainer() -> some View { modifier(FieldContainer()) }
func listRowNavy() -> some View { listRowBackground(AppColor.navySurface) }
}

```

Implementierungs-Hinweise (Views)

- View1Start:

- In der Etappen-ForEach-Zeile den VStack mit `.fieldContainer()` versehen.
- Für die Section selbst `listRowBackground` auf `AppColor.navySurface` setzen (oder `.listRowNavy()`).

• Keine systemgrauen Hintergründe mehr verwenden.

- Preview/QA:

- iOS → Settings → Appearance: Dark/Light prüfen.
- Kontraste checken (WCAG): Text auf navySurface $\geq 4.5:1$ (weiß erfüllt dies).